

# Real-Time Collaborative Workspace — Build Reference

## Project Goal

Build a production-style collaborative workspace where multiple users can work in the same workspace and see changes in real time.

The project should demonstrate:

- React
- FastAPI
- PostgreSQL
- SQLAlchemy
- REST APIs
- WebSockets
- Authentication
- Authorization
- Real-time state synchronization
- Concurrency handling
- Redis
- Docker
- Testing
- Deployment
- CI/CD

### Core principle:

Build the normal software system first. Add real-time functionality second. Add AI only after the core system is solid.

## 1. Product

Build a lightweight collaborative project-management application.

Users can:

- create an account
- log in
- create workspaces
- invite/add members

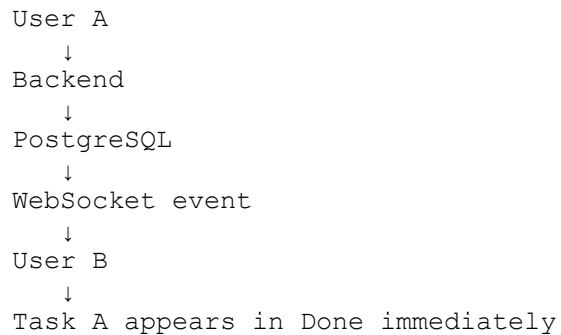
- create tasks
- edit tasks
- delete tasks
- move tasks between statuses
- comment on tasks
- see other users currently online
- see changes made by other users immediately
- handle simultaneous edits safely

## Main UI

A Kanban-style workspace:

|                                                     |                                             |                                      |
|-----------------------------------------------------|---------------------------------------------|--------------------------------------|
| Workspace: My Project <span>● 3 users online</span> |                                             |                                      |
| TODO<br><br>Task A<br>Task B<br><br>+ Add Task      | IN PROGRESS<br><br>Task C<br><br>+ Add Task | DONE<br><br>Task D<br><br>+ Add Task |

When User A moves Task A to Done:



## 2. Tech Stack

### Frontend

- React
- Vite
- JavaScript or TypeScript
- React Router
- Fetch/Axios

- CSS/Tailwind

## Backend

- Python
- FastAPI
- SQLAlchemy
- Pydantic
- JWT authentication
- WebSockets

## Database

- PostgreSQL

## Real-Time Infrastructure

- WebSockets
- Redis Pub/Sub

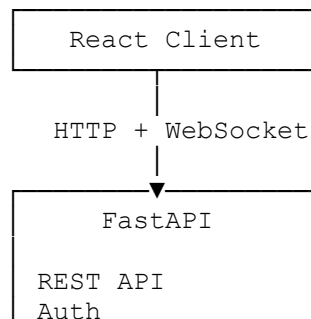
## Infrastructure

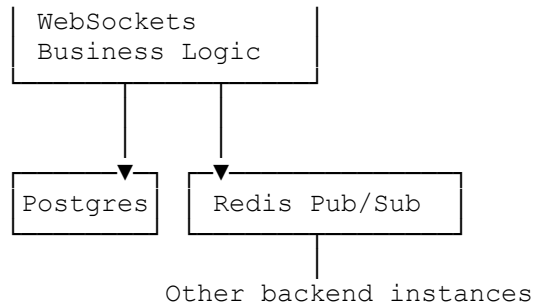
- Docker
- Docker Compose
- GitHub Actions

## Deployment

Choose simple deployment platforms rather than overengineering.

## 3. Architecture





## Important architectural rule

**PostgreSQL is the source of truth.**

React state is not the source of truth.

WebSockets are not the source of truth.

Redis is not the source of truth.

The flow is:

```
Client request
↓
Backend validation
↓
Database transaction
↓
Commit
↓
Create event
↓
Broadcast event
↓
Other clients update UI
```

## 4. Database Schema

### User

```
User
-----
id
email
password_hash
created_at
```

## Workspace

```
Workspace
-----
id
name
created_at
```

## WorkspaceMember

```
WorkspaceMember
-----
id
workspace_id
user_id
role
created_at
```

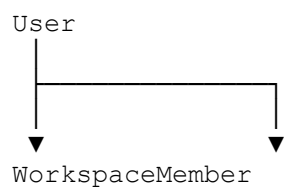
## Task

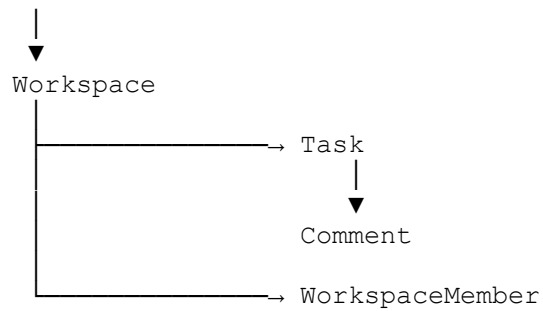
```
Task
-----
id
workspace_id
title
description
status
position
created_by
version
created_at
updated_at
```

## Comment

```
Comment
-----
id
task_id
user_id
body
created_at
```

## Relationships





A user can belong to multiple workspaces.

A workspace can contain multiple users.

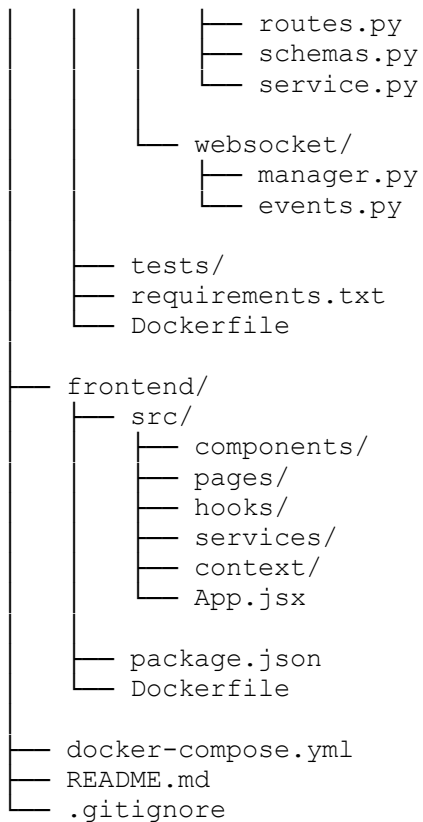
A workspace can contain multiple tasks.

A task can contain multiple comments.

## 5. Project Folder Structure

Start with something clean and evolve it as needed.

```
collabspace/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── config.py
│   │   ├── database/
│   │   │   ├── database.py
│   │   │   └── models/
│   │   ├── auth/
│   │   │   ├── routes.py
│   │   │   ├── service.py
│   │   │   └── dependencies.py
│   │   ├── workspaces/
│   │   │   ├── routes.py
│   │   │   ├── schemas.py
│   │   │   └── service.py
│   │   ├── tasks/
│   │   │   ├── routes.py
│   │   │   ├── schemas.py
│   │   │   └── service.py
│   │   └── comments/
```



Don't obsess over the exact folder structure. Keep responsibilities separated.

## 6. Development Sequence

Build in this order.

1. Project setup
2. Database
3. Authentication
4. Workspaces
5. Tasks
6. Basic frontend
7. WebSockets
8. Real-time synchronization
9. Presence
10. Comments
11. Concurrency handling
12. Redis
13. Testing
14. Docker
15. CI/CD
16. Deployment
17. AI feature

Do not jump ahead because a feature sounds cool.

## 7. Phase 1 — Project Setup

### Goal

Get the frontend and backend running independently.

### Backend

Create:

```
GET /health
```

Response:

```
{  
  "status": "ok"  
}
```

### Frontend

Create:

- React application
- basic routing
- basic layout
- API client

### Verify

Frontend loads.

Backend loads.

Frontend can call:

```
GET /health
```

## 8. Phase 2 — Database

Implement:

- PostgreSQL
- SQLAlchemy
- database connection
- models
- migrations if using Alembic

Create:

User  
Workspace  
WorkspaceMember  
Task  
Comment

Add:

- foreign keys
- indexes
- unique constraints
- relationships
- timestamps

## Important

Understand:

- primary key
- foreign key
- one-to-many
- many-to-many
- indexes
- transactions

# 9. Phase 3 — Authentication

Implement:

POST /auth/register  
POST /auth/login  
GET /auth/me

## Registration

email  
password

**Password:**

plain password  
↓  
password hashing  
↓  
database

Never store plaintext passwords.

## Login

email + password  
↓  
verify password  
↓  
JWT

## Protected request

Client  
↓  
Authorization: Bearer <JWT>  
↓  
FastAPI  
↓  
validate JWT  
↓  
identify user  
↓  
allow/deny request

# 10. Phase 4 — Workspace System

Implement:

```
POST    /workspaces
GET      /workspaces
GET      /workspaces/{id}
POST     /workspaces/{id}/members
DELETE   /workspaces/{id}/members/{user_id}
```

Rules:

- creator automatically becomes owner
- only members can access workspace

- only authorized users can modify membership
- users cannot access another user's private workspace

## 11. Phase 5 — Task System

Implement:

```
GET    /workspaces/{id}/tasks
POST   /workspaces/{id}/tasks
PATCH /tasks/{id}
DELETE /tasks/{id}
```

Task fields:

```
title
description
status
position
version
created_by
created_at
updated_at
```

Statuses:

```
TODO
IN_PROGRESS
DONE
```

## 12. Phase 6 — Frontend

Build:

### Login

```
Email
Password
Login
```

### Workspace dashboard

```
My Workspaces

+ Create Workspace
```

## Workspace

Workspace Name

TODO                      IN PROGRESS                      DONE

Task cards:

|                      |
|----------------------|
| Build authentication |
| Created by Pratham   |

Add:

- create task
- edit task
- delete task
- move task
- task detail view
- comments

## 13. Phase 7 — WebSockets

### Basic connection

Create:

```
WS /ws/workspaces/{workspace_id}
```

When user opens a workspace:

```
React
  ↓
WebSocket connection
  ↓
FastAPI
```

Maintain active connections by workspace.

Example:

```
workspace_1:
  user_A → socket_A
  user_B → socket_B
```

```
workspace_2:
  user_C → socket_C
  user_D → socket_D
```

## 14. WebSocket Connection Manager

Create a manager responsible for:

```
connect()
disconnect()
send_to_user()
broadcast_to_workspace()
```

Conceptually:

```
connections = {
  workspace_id: {
    user_id: websocket
  }
}
```

When a workspace event occurs:

```
broadcast_to_workspace(
  workspace_id,
  event
)
```

Only connected users in that workspace receive it.

## 15. WebSocket Events

Start with:

```
TASK_CREATED
TASK_UPDATED
TASK_DELETED
```

Example:

```
{
  "type": "TASK_UPDATED",
  "task": {
    "id": 42,
    "title": "Build API",
  }
}
```

```
    "status": "DONE",  
    "version": 3  
  }  
}
```

## 16. Real-Time Task Flow

When User A updates a task:



### Critical rule

**Never broadcast a successful-looking event before the database operation succeeds.**

Otherwise clients can display state that doesn't actually exist in the database.

## 17. Frontend WebSocket Handling

When workspace opens:

```
connect WebSocket
```

When event arrives:

```
TASK_CREATED
  ↓
add task to state

TASK_UPDATED
  ↓
replace/update task in state

TASK_DELETED
  ↓
remove task from state
```

Example:

```
WebSocket Event
  ↓
event.type
  ↓
switch(type)
  ↓
update React state
  ↓
UI rerenders
```

## 18. Reconnection

Handle:

- connection closed
- server unavailable
- network interruption
- browser reconnect

Display:

 Connected  
 Reconnecting...  
 Offline

Don't reconnect infinitely with no delay.

Use controlled retry/backoff.

## 19. Phase 8 — Presence

Show currently connected users.

Example:

Online now

● Pratham  
● Alex  
● Sarah

Events:

USER\_JOINED  
USER\_LEFT

When a user connects:

```
connect
↓
add user to workspace presence
↓
broadcast USER_JOINED
```

When disconnected:

```
disconnect
↓
remove user
↓
broadcast USER_LEFT
```

Presence should be **ephemeral**.

Don't store online status permanently in PostgreSQL.

## 20. Phase 9 — Comments

Implement:

```
GET /tasks/{task_id}/comments
POST /tasks/{task_id}/comments
```

Comment:

```
author
body
created_at
```

When a comment is created:

```
POST comment
  ↓
PostgreSQL
  ↓
commit
  ↓
COMMENT_CREATED
  ↓
WebSocket
  ↓
other clients
```

## 21. Phase 10 — Concurrency

This is one of the most important technical parts.

Problem:

```
User A reads Task 1
version = 4
```

```
User B reads Task 1
version = 4
```

A changes title.

B changes status.

Both try to save.

Without concurrency control:

```
A saves
B saves
```

B may accidentally overwrite A's change.

## 22. Optimistic Concurrency Control

Task contains:

version

Example:

Task version = 4

Client sends:

Update Task  
expected\_version = 4

Backend checks:

Current DB version == expected version?

**Yes**

Update:

version 4 → 5

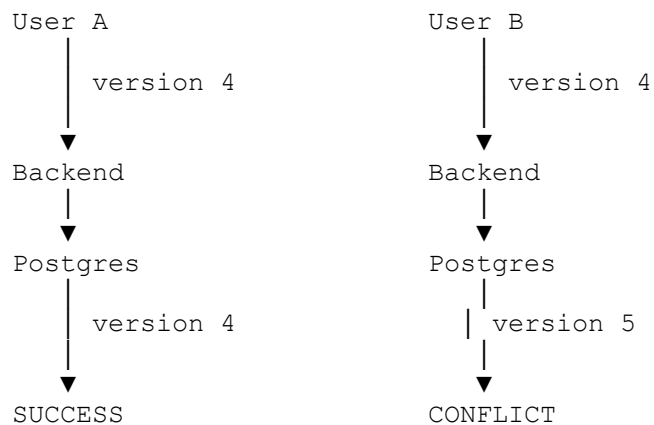
**No**

Reject:

409 Conflict

Return current task.

## 23. Conflict Flow



The frontend should tell the user that their version is outdated instead of silently overwriting newer data.

## 24. Important Interview Concept

Be able to explain:

**Why don't WebSockets solve concurrency?**

Because WebSockets only provide communication.

They don't automatically decide:

- which update wins
- whether an update is stale
- whether two changes conflict
- how the database should handle concurrent writes

WebSocket:

communication

Optimistic concurrency:

consistency

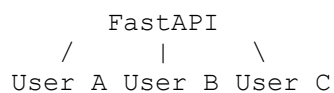
They solve different problems.

## 25. Phase 11 — Redis

Only add Redis **after the single-server version works.**

### Problem

With one backend:



Easy.

With multiple backend instances:

User A → Server 1

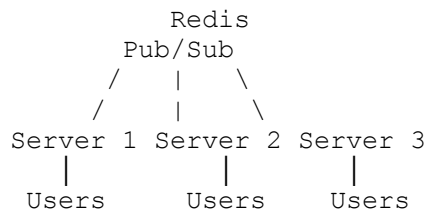
User B → Server 2

User C → Server 3

Server 1 doesn't automatically know about connections on Server 2 and Server 3.

## 26. Redis Pub/Sub

Architecture:



When a task changes:

```
Server 1
  ↓
Redis channel
  ↓
Server 2 + Server 3
  ↓
their WebSocket clients
```

PostgreSQL remains the source of truth.

Redis is for distributing real-time events between backend instances.

## 27. Docker

Create Docker containers for:

```
Frontend
Backend
PostgreSQL
Redis
```

Use:

```
docker-compose.yml
```

Goal:

```
docker compose up
```

should start the application stack.

## 28. Testing

Add backend tests for:

### Authentication

- register
- login
- invalid password
- invalid JWT

### Authorization

- workspace member access
- non-member rejection
- permission checks

### Tasks

- create
- update
- delete
- invalid task
- unauthorized task

### Concurrency

- valid version
- stale version
- conflict response

### WebSockets

- connection
- disconnect
- task event
- workspace broadcast

## 29. Test Types To Understand

### Unit test

Tests one isolated piece of logic.

Example:

```
password validation
```

### Integration test

Tests multiple components working together.

Example:

```
API
↓
SQLAlchemy
↓
PostgreSQL
```

### End-to-end test

Tests the system from the user's perspective.

Example:

```
Login
↓
Open workspace
↓
Create task
↓
Second browser sees task
```

## 30. CI/CD

Use GitHub Actions.

On every push:

```
GitHub
  ↓
GitHub Actions
  ↓
Install dependencies
  ↓
Run tests
  ↓
Build
```

Don't overcomplicate CI/CD.

The goal is to demonstrate that you understand automated testing/build pipelines.

## 31. Production Features

Before calling it finished, add:

- environment variables
- secure secrets
- error handling
- request validation
- logging
- database connection handling
- WebSocket disconnect handling
- loading states
- error states
- empty states
- responsive UI
- API documentation
- README

## 32. AI Feature — Optional Final Layer

**Do this last.**

The application should already be a good SWE project without AI.

Add one useful AI feature.

## **Option A — AI Task Breakdown**

User:

Build an e-commerce website

AI generates:

1. Design database
2. Implement authentication
3. Build product API
4. Build frontend
5. Add payment system

User can approve tasks and add them to the workspace.

## **Option B — Workspace Summary**

AI analyzes recent activity:

Today's Summary

- 7 tasks completed
- 3 tasks created
- Authentication API was completed
- Database migration is still pending
- Alex completed the frontend work

## **Option C — Task Assistant**

User asks:

What should I work on next?

AI considers:

- task status
- priority
- dependencies
- recent activity

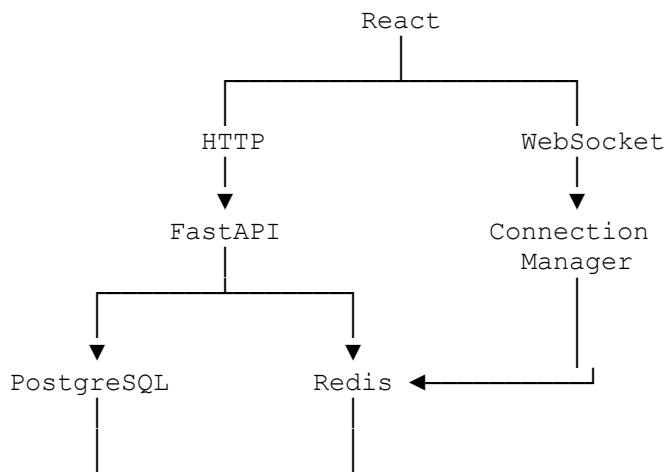
and recommends a task.

## Recommended

### Workspace Summary or AI Task Breakdown.

Keep AI as a feature, not the foundation.

## 33. Final Architecture



## 34. Final Feature Checklist

### Core

- User registration
- Login
- JWT authentication
- Workspace creation
- Workspace membership
- Authorization
- Task CRUD
- Kanban board
- Comments

### Real-Time

- WebSocket connection
- Workspace rooms

- Task-created events
- Task-updated events
- Task-deleted events
- Real-time frontend updates
- Reconnection
- Presence
- User joined events
- User left events

## Advanced

- Optimistic concurrency
- Conflict handling
- Redis Pub/Sub
- Multi-instance architecture

## Engineering

- PostgreSQL
- SQLAlchemy
- API validation
- Error handling
- Logging
- Automated tests
- Docker
- CI/CD
- Deployment
- README
- Architecture diagram

## Optional AI

- AI task breakdown OR
- AI workspace summary OR
- AI task assistant

# 35. Final README Structure

```
# CollabSpace
```

```
Real-time collaborative project management platform.
```

```
## Features
```

- Multi-user workspaces
- Real-time task synchronization
- WebSocket communication
- Presence tracking
- Optimistic concurrency control
- PostgreSQL persistence
- Redis Pub/Sub
- JWT authentication
- Docker deployment

## ## Architecture

[Architecture Diagram]

## ## Tech Stack

Frontend:  
React

Backend:  
FastAPI

Database:  
PostgreSQL

Real-Time:  
WebSockets + Redis

Infrastructure:  
Docker + GitHub Actions

## ## How It Works

[Explain HTTP + WebSocket architecture]

## ## Concurrency

[Explain version-based optimistic concurrency]

## ## Running Locally

[Instructions]

## ## Testing

[Instructions]

## ## Live Demo

[URL]

## ## Screenshots

[Images]

## 36. What You Should Be Able to Explain in an Interview

By the end, you should independently understand:

HTTP vs WebSockets

REST API design

JWT authentication

Authentication vs authorization

PostgreSQL relationships

Transactions

Database indexes

Connection pooling

WebSocket connection management

Real-time event architecture

Client/server state synchronization

Optimistic concurrency control

Race conditions

Redis Pub/Sub

Horizontal scaling

Docker

CI/CD

Unit vs integration testing

If you can't explain one of these, **don't hide behind the code. Stop and learn that concept.**

---

## 37. Definition of Done

The project is **not finished** when:

the UI looks good

It is finished when:

Two users can log in  
↓  
join the same workspace  
↓  
create/edit/delete tasks  
↓  
see each other's changes instantly  
↓  
see who is online  
↓  
comment in real time  
↓  
handle conflicting edits  
↓  
run the entire stack with Docker  
↓  
tests pass  
↓  
application is deployed

Then you have something worth putting prominently on your resume.

## Resume positioning

Don't write:

"Built a collaborative task management app."

Aim for something closer to:

**Built a real-time collaborative workspace supporting multi-user task synchronization through FastAPI WebSockets and PostgreSQL, with JWT authentication, workspace-based authorization, presence tracking, and optimistic concurrency control.**

And if Redis/multi-instance support is actually implemented:

**Designed Redis Pub/Sub event distribution to synchronize WebSocket clients across multiple backend instances.**

Only claim what you actually built.